CS311

Introduction to Web Services Development

Complete Notes — Module 1 to Module 10

Virtual University of Pakistan

Prepared by:

BEYZA AYSUN

MODULE 1: Object Oriented Paradigm

Objective: This module covers Object Orientation in detail, including Data Modeling, Abstraction, Inheritance, Encapsulation, Information Hiding, and their use in classes.

What is Object Orientation?

In the real world, the object-oriented approach focuses on objects that represent abstract or concrete things. Objects are the basic building blocks of OOP.

What is an Object?

Objects are key to understanding object-oriented programming. Look around yourself and you'll find many real-world examples: your dog, your desk, your TV, your bicycle. An object has two key characteristics:

- State — a noun, attribute, or well-defined property (e.g., dog's colour, name, breed)
- Behavior — a certain operation or action (e.g., dog's barking, fetching)

> **Note:** Object = Variables + Functions. Example: Ali, House, Car are objects. Relation: Ali lives in the house, Ali drives a car.

What is a Model?

Modeling is an approach used at the beginning of software development. Reasons to model:

- Communication: Diagrams before implementation help users give feedback on system structure.
- Abstraction: Define what functionality is needed, then refine into an implementable design.

What is Abstraction?

Abstraction is the process of taking out essential characteristics from something in order to reduce it to a set of key features. A programmer hides irrelevant data about an object to reduce complexity and increase efficiency.

Graphical Class Representation:

- Class name at the top
- Attributes in the middle
- Operations/methods at the bottom
- + (plus) = public, − (minus) = private, # = protected

Advantages of Abstraction:

- Separate things that can be grouped to another type

- Frequently changing properties can be grouped separately
- Simplifies the representation of domain models

What is Inheritance?

Inheritance enables new objects to take on the properties of existing objects.

- Superclass / Base Class / Parent Class — the class from which properties are inherited
- Subclass / Derived Class / Child Class — the class that inherits
- A child inherits all characteristics of its parent and can add additional ones

Superclass (Parent)	Subclass (Child)
Also called Base Class	Also called Derived Class
Provides common properties	Inherits + adds own properties
Example: Animal	Example: Dog, Cat
Example: Fruit	Example: Orange, Apple
Example: Vehicle	Example: Car, Bike

Types of Inheritance

Type	Description	Example
Single Inheritance	One parent → One child	Animal → Dog
Multilevel Inheritance	Grandparent → Parent → Child	A → B → C
Multiple Inheritance	Multiple parents → One child	A, B → C
Hierarchical	One parent → Multiple children	Animal → Cat, Dog

Generalization & Specialization

Generalization	Specialization
Combining common features	Creating specific classes from general
Bottom-up approach	Top-down approach
Example: Cat + Dog → Animal	Example: Animal → Cat

Encapsulation & Information Hiding

Encapsulation is a way to obtain information hiding — you don't need to know the internal working of a mobile phone to operate it. You have an interface to use the device behavior without knowing implementation details.

- Encapsulation = bundling data and methods together
- Information Hiding = hiding sensitive data from outside access

Access Modifier	Accessibility
public	Accessible from anywhere
private	Only within the same class
protected	Same class + subclasses
default (no modifier)	Same package only

Relationships: Association, Aggregation & Composition

Composition	Aggregation
Strong whole-part relationship	Weak relationship
Child cannot exist without parent	Child can exist independently
Parent deleted → Child deleted	Parent deleted → Child survives
Example: House has Rooms	Example: Department has Teachers

Abstract Class vs Interface

Abstract Class	Interface
Can have some implementation	Only method declarations (Java < 8)
Uses extends keyword	Uses implements keyword
'is-a' relationship	Contract / Service agreement
Can have constructors	Cannot have constructors

Overloading vs Overriding

Overloading	Overriding
Same class	Parent-child classes
Different parameters	Same parameters
Compile-time polymorphism	Runtime polymorphism

Overloading	Overriding
Example: add(int), add(double)	Child.eat() overrides Parent.eat()

Polymorphism

Polymorphism means 'one interface, many forms'. It allows one method to behave differently based on the object.

- Up-casting: Child object → Parent type (Automatic, Safe, Implicit)
- Down-casting: Parent object → Child type (Manual, Unsafe, Explicit)
- Dynamic Binding: The method to be called is determined at runtime

MODULE 2: Basics of Java Programming

This module covers Java variable types, JVM, data types, and operators.

Types of Variables in Java

Feature	Local Variable	Instance Variable
Where declared	Inside methods/blocks	Inside class, outside method
Memory	Stack	Heap
Default value	None (must initialize)	Yes (0, false, null)
Access modifier	Not allowed	Allowed
Lifetime	Till method ends	Till object is destroyed

Static (Class) Variables

Static variables are declared with the static keyword in a class, outside any method or block.

- Only one copy per class — shared by all objects
- Created when program starts, destroyed when program stops
- Accessed as: ClassName.VariableName
- Constant static variables are named in UPPER_CASE

JVM — Java Virtual Machine

JVM (Java Virtual Machine) is an abstract machine that provides the runtime environment in which Java bytecode can be executed.

- Runs Java programs across different hardware/software platforms
- Converts bytecode to machine code
- Manages memory and performs Garbage Collection
- Makes Java platform-independent

Data Types

Primitive Types	Non-Primitive Types
int, char, float, double	String, Array, Class
Stored in Stack memory	Stored in Heap memory
Hold direct values	Hold references (addresses)
Fixed size	Variable size

Arithmetic Operators

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (Remainder)
++	Increment (add 1)
--	Decrement (subtract 1)

Variable Types Summary

Variable Type	Real-Life Analogy
Local Variable	Rough work on exam paper — temporary
Instance Variable	Each student's own name & roll number — different for each
Static Variable	School name — same for every student

MODULE 3: Program Control Flow

Control flow statements break up the normal top-to-bottom execution by using decision making, looping, and branching.

Selection Statements

if Statement

Executes a block of code only if a condition is true.

```
if (condition) {
    // code executes when condition is true
}
```

if-else Statement

Provides a secondary path when the 'if' condition is false.

```
if (condition) {
    // true path
} else {
    // false path
}
```

Nested if

An if statement inside another if statement.

switch-case Statement

Used to execute alternate blocks of code based on the value of a single variable.

```
switch (variable) {
    case value1:
        // code
        break;
    case value2:
        // code
        break;
    default:
        // code
}
```

Looping Statements

Loop Type	When to Use	Syntax
for loop	When repetitions count is known	for(init; condition; update)
while loop	When condition is unknown	while(condition) { }
do-while loop	Executes at least once	do { } while(condition);

Branching Statements

- break — exits loop or switch immediately
- continue — skips current iteration, continues loop
- return — exits from a method

Summary Table

Topic	Meaning
Nested If	if ke andar if
Local Block	{ } ke andar code
Local Scope	Variable sirf block mein accessible
Semicolon	Statement end karta hai
Sentinel Loop	Unknown repetitions wali loop
Sentinel Variable	Loop stop karne wala variable

MODULE 4: Using Objects

This module covers the Object Model, classes, objects, constructors, and memory management in Java.

Object Model

A collection of objects or classes through which a program can examine and manipulate parts of its world. Examples: DOM (Document Object Model) for web browsers, Excel Object Model for controlling Excel.

Object Reference

Objects are accessed via object references. To invoke a method, the object reference and method name are given along with arguments.

> **Note:** Class does NOT occupy memory. When you create an object using 'new', memory is allocated in the Heap.

Classes — Blueprint of Objects

A class contains variables for storing data and functions for performing operations. It is only a logical representation until an object is created from it.

```
class Employee {
    String name;
    int id;
    void display() {
        System.out.println(name + " " + id);
    }
}
```

Object Creation

```
Employee emp = new Employee(); // creates object in Heap memory
```

- new keyword allocates memory in Heap
- Reference variable (emp) is stored in Stack
- Reference variable holds the address of the object

Constructors

Default Constructor	Parameterized Constructor
No parameters	Takes parameters
Provided automatically if none defined	Must be defined manually
Sets default values	Initializes with given values
Employee()	Employee(String name, int id)

Pass by Value

In Java, all variables are passed by value. This means a copy of the value is passed — the original variable is not changed.

- Primitive types: copy of value is passed
- Objects/Arrays: copy of reference is passed (but original object can be modified)

Garbage Collector

Java's Garbage Collector automatically removes objects from Heap memory when they are no longer referenced. This prevents memory leaks.

MODULE 5: Primitive Values as Objects

This module covers Wrapper Classes, Autoboxing/Unboxing, and Assignment Operators.

Wrapper Classes

A wrapper class encapsulates a single, immutable primitive value inside an object.

Primitive Type	Wrapper Class
int	Integer
float	Float
double	Double
char	Character
boolean	Boolean
long	Long
byte	Byte
short	Short

Autoboxing & Unboxing

Autoboxing is the automatic conversion from primitive types to their corresponding Wrapper class objects. The reverse is called Unboxing.

```
Integer iRef = 10;  // Autoboxing: int → Integer
int j = iRef;       // Unboxing: Integer → int
```

Autoboxing	Unboxing
Primitive → Object	Object → Primitive
Automatic	Automatic
int → Integer	Integer → int

Extending Assignment Operators

Operator	Meaning
+=	Add and assign
-=	Subtract and assign
*=	Multiply and assign
/=	Divide and assign

Operator	Meaning
%=	Modulus and assign
&=	Logical AND and assign
|=	Logical OR and assign

String Class

String is an immutable sequence of characters. Java stores string literals in a String Pool to save memory.

Method	Purpose
length()	Returns number of characters
charAt(i)	Returns character at index i
concat(str)	Joins two strings
equals(str)	Compares content (use this!)
== operator	Compares references (not content)
compareTo(str)	Alphabetical comparison
toUpperCase()	Converts to uppercase
toLowerCase()	Converts to lowercase

MODULE 6: Arrays

An array is an object containing a list of elements of the same data type, stored in contiguous memory locations.

Declaring and Creating Arrays

```
// Step 1: Declare array reference variable
double[] dailySales;

// Step 2: Create array object
dailySales = new double[7];

// Combined in one statement
double[] dailySales = new double[7];
```

Initializing an Array

```
int[] num = {10, 20, 30, 40, 50}; // Direct initialization
int[] arr = new int[5];           // Size-based (default = 0)
```

Accessing Array Elements

Array elements are accessed using index, starting from 0.

```
dailySales[0] = 100.5; // First element
dailySales[6] = 200.0; // Last element (size 7)
```

> **Note:** Array index starts at 0. For an array of size n, valid indices are 0 to n-1.

Multidimensional Arrays

An array of arrays — useful for matrices and tables.

```
int[][] matrix = new int[3][3]; // 3x3 matrix
matrix[0][0] = 1;
```

Random Class

The Random class is used to generate random numbers.

```
import java.util.Random;
Random rand = new Random();
int num = rand.nextInt(100); // Random number between 0-99
```

Nested Loops with Arrays

Loops are commonly used with arrays to access all elements.

```
for (int i = 0; i < arr.length; i++) {
```

```
    System.out.println(arr[i]);
}
```

MODULE 7: Classes

This module covers methods, access modifiers, this keyword, constructors, and class design in Java.

Methods

A method is a reusable block of code that performs a specific task. Methods have:

- Access Modifier — controls visibility
- Return Type — what the method returns (void = nothing)
- Method Name — identifier
- Parameters — input values
- Method Body — the code block

```
public int add(int a, int b) {
    return a + b;
}
```

Types of Methods

Type	Purpose	Example
Mutator (setter)	Changes a field value	setName(String n)
Selector (getter)	Reads a field value	getName()
Utility/Helper	Performs a helper task	calculateTax()
Static method	Belongs to class, not object	Math.sqrt()

Access Modifiers

Modifier	Accessibility
public	Accessible from anywhere
private	Only within the same class
protected	Same class + subclasses (other packages too)
default (no modifier)	Only within the same package

Field Variables vs Local Variables

Field (Instance) Variable	Local Variable
Declared inside class, outside methods	Declared inside methods/blocks

Field (Instance) Variable	Local Variable
Accessible to all methods in class	Only accessible in its scope
Has a default value	No default — must initialize
Lives as long as the object	Lives only during method execution

The 'this' Keyword

'this' refers to the current object. It is used to differentiate between field variables and local/parameter variables with the same name.

```
class Student {
    String name;
    Student(String name) {
        this.name = name; // 'this.name' = field, 'name' = parameter
    }
}
```

Formal vs Actual Parameters

Formal Parameter	Actual Parameter
Variable in method definition	Value passed during method call
int add(int a, int b)	add(5, 10)
Placeholder/template	Real value

Static vs Instance

Static	Instance
Belongs to the class	Belongs to each object
Shared by all objects	Each object has its own copy
Access: ClassName.field	Access: objectName.field
Example: school name	Example: student name

MODULE 8: Object Communication

Objects communicate with each other by calling each other's methods. This module covers structured programming methodology and how objects interact.

Structured Programming

Structured programming promotes organized, readable, and maintainable code using blocks, conditions, and loops.

Block

A block is a section of code grouped together with { }. Blocks can be nested inside each other.

```
{
    int x = 10; // local to this block
    System.out.println(x);
}
```

Object Communication Overview

Objects communicate by invoking methods on each other. One object can hold a reference to another and call its methods.

```
class Engine {
    void start() { System.out.println("Engine started!"); }
}
class Car {
    Engine engine = new Engine();
    void drive() { engine.start(); }
}
```

Abstraction Principles

- Use abstractions whenever suitable to avoid code duplication
- Pull out common pieces of functionality into reusable components
- Abstract classes, parent classes, and interfaces are all forms of abstraction
- Abstraction = representing an object before its implementation

Association Types

Association Type	Description
One-Way Association	Relation in one direction only
Two-Way Association	Both objects know each other
Ternary Association	Three entities related together
Aggregation	Weak whole-part (child independent)

Association Type	Description
Composition	Strong whole-part (child dependent)

Summary Table

Topic	Meaning
Object Communication	Objects call each other's methods
this	Current object reference
Reference Variable	Holds object address
Association	Relationship between classes
Aggregation	Weak relationship
Composition	Strong whole-part relationship
Structured Programming	Organized, maintainable code

MODULE 9: Modifiers

Java has two types of modifiers: Access Modifiers and Non-Access Modifiers.

Access Modifiers

Access modifiers control the accessibility (scope) of a class, method, field, or constructor.

Modifier	Accessible From
public	Everywhere — any class, any package
protected	Same package + subclasses in other packages
default	Same package only (no keyword needed)
private	Only within the same class

public

A public class, method, or field can be accessed from any other class. The main() method must be public for the JVM to call it.

```
public static void main(String[] args) {
    // entry point of program
}
```

protected

Protected members can be accessed by subclasses (even in other packages) and by all classes within the same package.

```
class AudioPlayer {
    protected boolean openSpeaker(Speaker sp) {
        // implementation
    }
}
```

private

Private members are only visible within the same class. Best practice: declare fields private, provide public getter/setter methods.

Non-Access Modifiers

Modifier	Meaning
static	Belongs to class, not instance. Shared by all objects.

Modifier	Meaning
final	Cannot be changed (variable) or overridden (method) or extended (class)
abstract	Incomplete class/method — must be implemented by subclass
synchronized	Only one thread can access at a time (thread safety)
native	Implemented in another language (e.g., C/C++)
transient	Not saved during serialization
volatile	Latest value visible to all threads — used in multi-threading

final Keyword

- final variable: constant — cannot be reassigned
- final method: cannot be overridden by subclasses
- final class: cannot be extended (e.g., String class is final)

```
final int MAX = 100;  // constant
final class MyClass { }  // cannot extend
```

abstract Keyword

An abstract class cannot be instantiated directly. It may contain abstract methods (no body) that subclasses must implement.

```
abstract class Shape {
    abstract double area(); // no body — subclass must implement
}
```

MODULE 10: Exception Handling

An exception is a problem that arises during program execution that disrupts the normal flow. Java provides a robust mechanism to handle exceptions gracefully.

What is an Exception?

Common reasons exceptions occur:

- User has entered invalid data
- A required file cannot be found
- Network connection lost during communication
- JVM has run out of memory

Types of Exceptions

Type	When	Examples
Checked Exception	Compile time — must be handled	IOException, SQLException
Unchecked Exception	Runtime — programming errors	ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException
Error	Severe system problems — not handleable	StackOverflowError, OutOfMemoryError

Exception Hierarchy

All exceptions extend the Throwable class:

- Throwable
  - → Exception (handleable problems)
    - → IOException (checked)
    - → RuntimeException (unchecked)
  - → Error (severe — not handled)

try-catch-finally Block

```
try {
    // risky code that may throw exception
} catch (ExceptionType e) {
    // handle the exception
} finally {
    // always executes — cleanup code
}
```

Important Rules

- A catch clause cannot exist without a try statement
- A try block must have at least a catch or a finally block
- The finally block always executes regardless of exception
- No code can exist between try, catch, and finally blocks

throw and throws Keywords

throw	throws
Used inside a method body	Used in method signature
Manually throws an exception	Declares that method may throw exception
throw new IOException()	public void read() throws IOException
Throws a single exception	Can declare multiple exceptions

```
// throw example
public void deposit(double amount) throws RemoteException {
    throw new RemoteException();
}

// Multiple exceptions with throws
public void withdraw(double amount)
        throws RemoteException, InsufficientFundsException {
    // method body
}
```

Summary Table

Topic	Meaning
Exception	Runtime problem/error
Checked Exception	Compile time exception — must handle
Unchecked Exception	Runtime exception
Error	Serious system problem
try	Wraps risky code
catch	Handles the exception
finally	Always executes (cleanup)
throw	Manually throw an exception
throws	Declare possible exceptions in method signature

— End of Notes: Modules 1–10 —

Prepared by BEYZA AYSUN | CS311 — Introduction to Web Services Development